Program to an Interface, Coupling–Cohesion, dan Pemodelan UML



Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-3 | Sub-CPMK12
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan

Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	Prinsip perancangan berorientasi objek
03	**Interface, coupling–cohesion, UML**
04	SOLID bagian 1 — SRP, OCP, LSP
05	SOLID bagian 2 — ISP, DIP
06	Code smell & refactoring
07	Klaster 1 — kontrol pembuatan objek
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pertemuan ini menutup Sub-CPMK12 dan meluncurkan Tugas 1

12 Sub-CPMK12

Mampu menjelaskan prinsip perancangan berorientasi objek dan memodelkannya dalam diagram UML.

Bobot pada RPS: 12,50% dari nilai akhir.

S Bentuk penilaian pekan ini

Sumatif — tugas pemodelan (LK01).

Bobot: 4,00% melalui TUGAS1.

Diumumkan pekan ini, dikumpulkan akhir pekan ke-4.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Empat pokok bahasan, bersumber dari [1], [2], dan P1

1 Program to an interface

Klien bergantung pada tipe abstrak, bukan pada kelas konkret.

2 Coupling dan cohesion

Dua ukuran mutu rancangan yang saling melengkapi.

3 Pemodelan UML

Class diagram dan sequence diagram beserta notasinya.

4 Penjelasan Tugas 1

Lingkup, luaran, kriteria penilaian, dan jadwal LK01.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Menjelaskan prinsip program to an interface, not an implementation beserta akibatnya bagi ketergantungan antarkelas.
- Menganalisis coupling dan cohesion pada sebuah rancangan serta mengenali gejalanya pada kode program.
- Memodelkan rancangan ke dalam class diagram dengan notasi relasi dan multiplisitas yang benar.
- Memodelkan skenario penggunaan ke dalam sequence diagram yang konsisten dengan class diagram.
- Menerjemahkan class diagram menjadi kerangka kelas Java.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Program to an Interface, Not an Implementation

Bagian 1 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Makna Kata “Interface”

Prinsip ini berbicara tentang makna pertama; kata kunci hanyalah salah satu cara mewujudkannya

Antarmuka sebagai konsep

- **Himpunan operasi.** Kumpulan pesan yang boleh dikirimkan kepada sebuah objek.
- **Kontrak perilaku.** Bukan sekadar tanda tangan metode, melainkan juga janji apa yang terjadi.
- **Tidak selalu kata kunci.** Kelas abstrak pun menyediakan antarmuka bagi kliennya.

Antarmuka sebagai kata kunci Java

- **Tipe tanpa implementasi.** Menyatakan operasi tanpa menetapkan caranya.
- **Banyak implementasi.** Satu kelas dapat memenuhi lebih dari satu antarmuka.
- **Alat, bukan tujuan.** Menulis interface tidak otomatis membuat rancangan menjadi baik.

◆ **Intinya: yang dituntut prinsip ini adalah bergantung pada kontrak, bukan pada kelas tertentu.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Bergantung pada Kelas Konkret versus pada Antarmuka

Perhatikan tipe atribut dan tempat objeknya dibuat

Bergantung pada implementasi

```
class LayananKrs {
    private MySqlKrsRepository repo
            = new MySqlKrsRepository();

    public void simpan(Krs krs) {
        repo.insert(krs);
    }
}

// Ganti basis data  -> kelas ini ikut diubah.
// Uji tanpa basis data -> tidak mungkin.
```

Bergantung pada antarmuka

```
interface KrsRepository {
    void simpan(Krs krs);
}

class LayananKrs {
    private final KrsRepository repo;

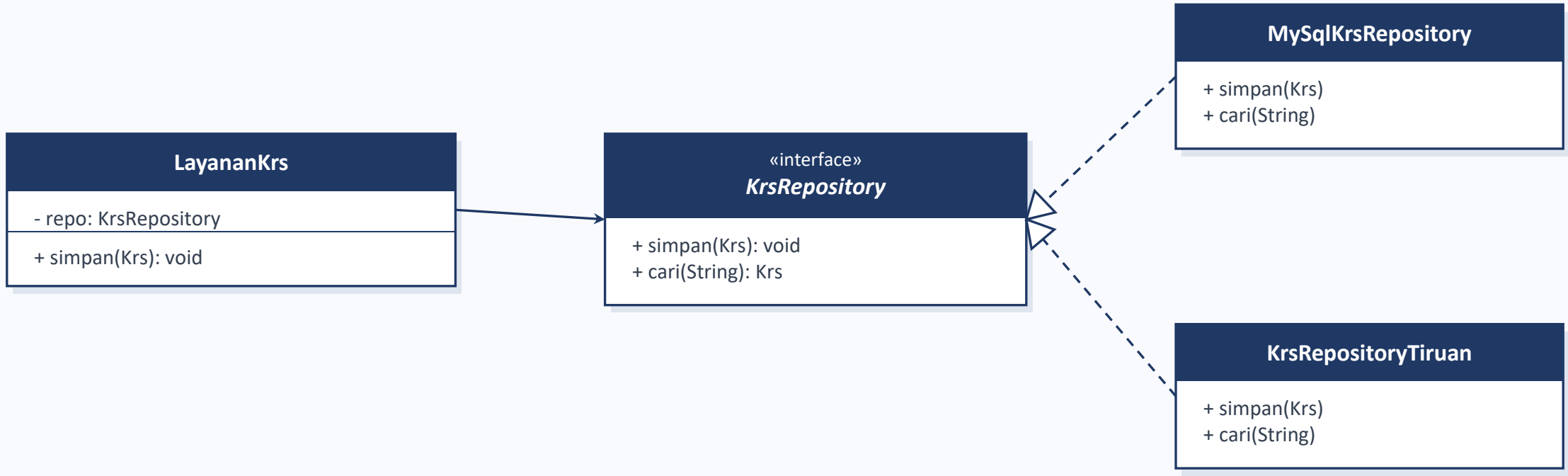
    public LayananKrs(KrsRepository repo) {
        this.repo = repo;   // dikirim dari luar
    }
    public void simpan(Krs krs) {
        repo.simpan(krs);
    }
}
```

- **Kelas kiri menciptakan sendiri objek yang dipakainya; kelas kanan menerimanya dari luar.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Arah Ketergantungan Setelah Prinsip Diterapkan

Aturan bisnis tidak lagi mengenal rincian teknis penyimpanan



- **Objek tiruan pada pengujian menjadi mungkin justru karena klien hanya mengenal antarmuka.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Manfaat dan Konsekuensi

Prinsip ini bukan tanpa ongkos

+ Implementasi bebas berganti

Selama kontrak dipenuhi, klien tidak perlu diubah sama sekali.

+ Pengujian menjadi mudah

Implementasi tiruan cukup memenuhi antarmuka yang sama.

+ Kesiapan bagi pola

Hampir seluruh pola GoF menuntut klien bergantung pada tipe abstrak.

– Objek harus dirakit

Ada pihak lain yang perlu memutuskan implementasi mana yang dipakai.

– Berkas bertambah

Satu abstraksi berarti satu berkas tambahan yang harus dipelihara.

– Abstraksi tanpa alasan

Antarmuka dengan satu implementasi tetap dan tanpa rencana perubahan justru mubazir.

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Coupling dan Cohesion

Bagian 2 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Ukuran Mutu Rancangan

Keduanya diukur pada arah yang berbeda dan harus dinilai bersama-sama

Coupling — antarmodul

- **Definisi.** Derajat saling ketergantungan antara satu modul dan modul lainnya.
- **Yang diinginkan.** Rendah — perubahan pada satu modul tidak merambat ke modul lain.
- **Gejala tinggi.** Mengubah satu kelas memaksa banyak kelas lain ikut diubah.

Cohesion — dalam satu modul

- **Definisi.** Derajat keterkaitan antartanggung jawab di dalam satu modul.
- **Yang diinginkan.** Tinggi — semua isi modul melayani satu tujuan yang sama.
- **Gejala rendah.** Satu kelas mengurusi hal-hal yang tidak berhubungan.

Intinya: rancangan yang baik memiliki coupling rendah dan cohesion tinggi sekaligus.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tingkatan Coupling, dari yang Terburuk ke Terbaik

Semakin ke bawah, semakin sedikit yang perlu diketahui satu modul tentang modul lainnya

Tingkat	Ciri	Penilaian
Content coupling	Satu modul mengubah langsung isi modul lain	Hindari
Common coupling	Beberapa modul berbagi data global yang sama	Hindari
Control coupling	Modul mengirim penanda yang mengatur alur modul lain	Kurangi
Stamp coupling	Modul mengoper struktur data utuh padahal hanya sebagian dipakai	Dapat diterima
Data coupling	Modul hanya mengoper data yang benar-benar diperlukan	Dianjurkan

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tingkatan Cohesion, dari yang Terburuk ke Terbaik

Cohesion fungsional adalah bentuk yang dituju oleh Single Responsibility Principle

Tingkat	Dasar pengelompokan	Penilaian
Coincidental	Tidak ada dasar; sekadar dikumpulkan	Hindari
Logical	Sejenis secara logika, dipilih lewat penanda	Hindari
Temporal	Kebetulan dijalankan pada waktu yang sama	Kurangi
Communicational	Mengolah data yang sama	Dapat diterima
Functional	Seluruh isinya melayani satu tugas tunggal	Dianjurkan

Single Responsibility Principle dibahas pada pekan ke-4 sebagai perumusan ulang cohesion menurut aktor.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Coupling Tinggi pada Kode

Satu kelas, empat arah ketergantungan, empat alasan untuk berubah

LaporanKrs.java

```java
class LaporanKrs {
    void cetak(Mahasiswa mhs) {
        // 1. tahu cara membuka koneksi   -> bergantung pada teknologi basis data
        Connection c = DriverManager.getConnection(URL, USER, PASS);

        // 2. tahu struktur tabel         -> bergantung pada skema penyimpanan
        ResultSet rs = c.createStatement().executeQuery("SELECT * FROM krs ...");

        // 3. tahu format keluaran        -> bergantung pada pustaka pencetakan
        PdfWriter w = new PdfWriter(new File(mhs.nim() + ".pdf"));

        // 4. tahu aturan akademik        -> bergantung pada kebijakan prodi
        if (mhs.ipk() < 2.0) w.tulis("berstatus percobaan");
    }
}
```

Empat sumbu perubahan berkumpul di satu kelas — coupling tinggi sekaligus cohesion rendah.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan: Menilai Sebuah Rancangan

Sepuluh menit berpasangan, lalu dibahas bersama

1 Baca kode pada slide sebelumnya

Tandai setiap baris yang menghubungkan kelas ini dengan dunia luar.

2 Kelompokkan menurut alasan berubah

Berapa kelompok yang Anda temukan? Siapa pemangku kepentingan tiap kelompok?

3 Usulkan pemisahan

Kelas apa saja yang perlu dibentuk, dan antarmuka apa yang menghubungkannya?

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Pemodelan Class Diagram dan Sequence Diagram

Bagian 3 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Notasi Kelas pada Class Diagram

Tiga kompartemen: nama, atribut, dan operasi

1 Nama kelas

Ditulis di kompartemen paling atas.

Miring bila kelas bersifat abstrak.

Stereotipe «interface» untuk antarmuka.

2 Atribut

Format: visibilitas nama: tipe.

Visibilitas: + publik, – privat, # terlindung.

Garis bawah menandai atribut kelas (static).

3 Operasi

Format: visibilitas nama(parameter): kembalian.

Miring bila operasi bersifat abstrak.

Cukup tulis operasi yang penting bagi rancangan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Lima Relasi yang Wajib Dibedakan

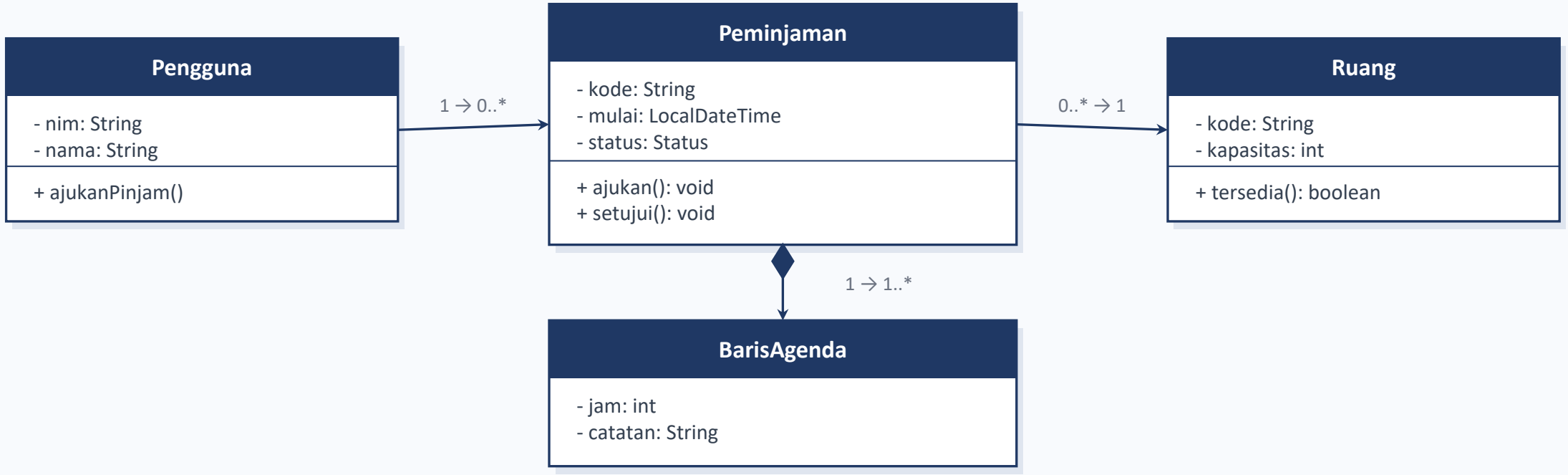
Kesalahan paling sering pada LK01 adalah menukar agregasi dengan komposisi

Relasi	Notasi garis	Makna	Contoh
Generalisasi	Garis utuh, ujung segitiga kosong	“adalah sebuah”	Dosen → Pegawai
Realisasi	Garis putus, ujung segitiga kosong	memenuhi kontrak	MySqlRepo ⇢ Repository
Asosiasi	Garis utuh polos atau berpanah	saling mengenal	Mahasiswa — MataKuliah
Agregasi	Garis utuh, belah ketupat kosong	“memiliki”, bagian dapat berdiri sendiri	Prodi ◇— Dosen
Komposisi	Garis utuh, belah ketupat penuh	“terdiri atas”, bagian ikut musnah	Krs ◆— BarisKrs

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh Class Diagram: Peminjaman Ruang Laboratorium

Perhatikan multiplisitas pada setiap ujung relasi



BarisAgenda tidak dapat hidup tanpa Peminjaman, karena itu relasinya komposisi.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Membaca Multiplisitas

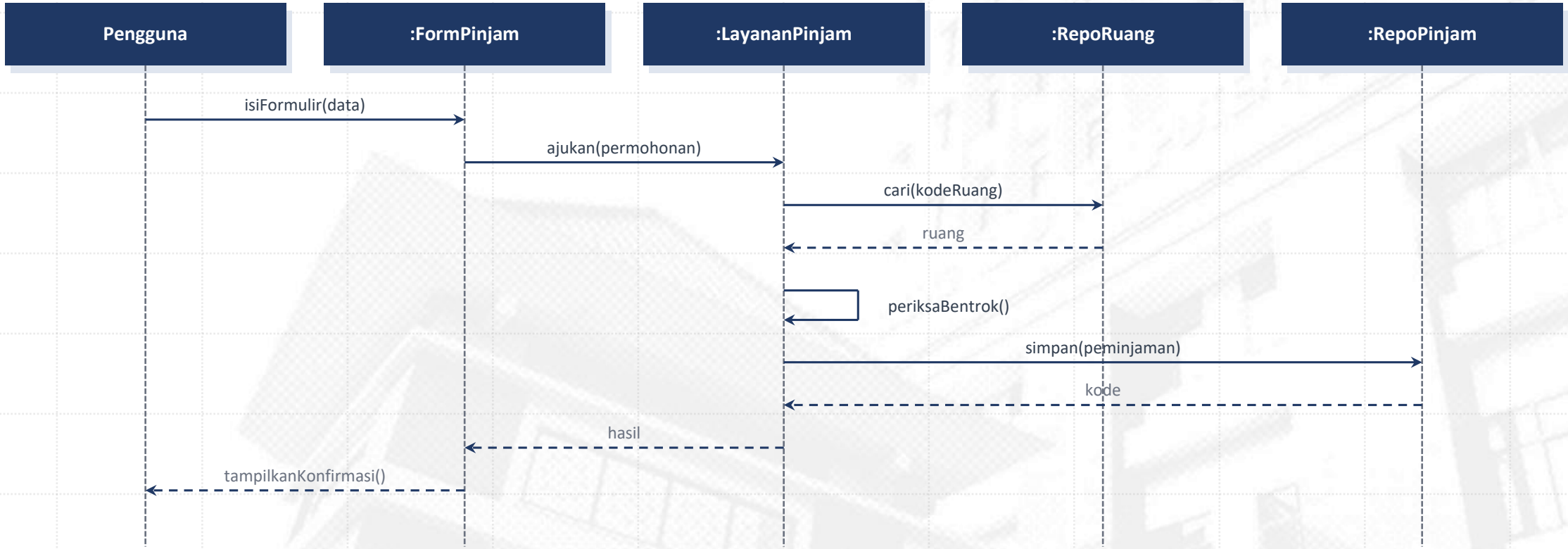
Multiplisitas adalah bagian rancangan yang paling sering terlupakan

Penulisan	Dibaca	Akibat pada kerangka kode Java
1	tepat satu	Atribut bertipe tunggal dan tidak boleh null
0..1	paling banyak satu	Atribut tunggal yang boleh kosong
1..*	sekurang-kurangnya satu	List yang dijamin tidak pernah kosong
0..* atau *	berapa pun, boleh kosong	List yang boleh kosong
2..5	antara dua dan lima	List dengan pemeriksaan jumlah pada konstruktor

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh Sequence Diagram: Skenario Pengajuan Peminjaman

Satu skenario utama, dibaca dari atas ke bawah



- **Setiap pesan pada diagram ini harus ada sebagai operasi pada class diagram.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Menjaga Konsistensi Kedua Diagram

Kriteria penilaian LK01 memeriksa kesesuaian ini secara khusus

Yang harus cocok

- **Nama kelas.** Setiap lifeline merujuk kelas yang ada pada class diagram.
- **Nama operasi.** Setiap pesan adalah operasi yang tercantum pada kelas penerima.
- **Arah rujukan.** Pengirim hanya dapat memanggil kelas yang dikenalnya melalui relasi.
- **Tipe kembalian.** Nilai balik sesuai dengan tanda tangan operasi.

Kesalahan yang sering terjadi

- **Pesan tanpa operasi.** Panggilan digambar padahal kelas tujuan tidak punya operasi itu.
- **Kelas tanpa relasi.** Objek saling memanggil padahal tidak ada asosiasi di antara keduanya.
- **Lifeline berupa aktor ganda.** Pengguna digambar sebagai kelas, bukan sebagai aktor.
- **Terlalu rinci.** Setiap getter digambar, sehingga alur utama tenggelam.

© Fakultas Ilmu Komputer, Universitas Brawijaya

04

MATERI KE-4

Tugas 1 — Pemodelan Rancangan dengan Diagram UML

Bagian 4 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

LK01 — Lingkup Pekerjaan

Tugas individu, mengukur Sub-CPMK12 dengan bobot 4,00%

1 Masukan

Deskripsi tekstual sebuah sistem sederhana beserta daftar kebutuhan fungsionalnya.

Dibagikan pada pertemuan ini.

2 Yang dikerjakan

Identifikasi kelas, atribut, operasi, dan relasi.

Penyusunan class diagram dan sequence diagram.

Penentuan bagian yang memakai komposisi atau pewarisan beserta alasannya.

3 Luaran

Laporan PDF berisi kedua diagram dan justifikasi.

Berkas sumber diagram (StarUML atau PlantUML).

Kerangka kelas Java pada repositori Git.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Metode Pengerjaan yang Dianjurkan

Enam langkah sesuai Lembar Kerja LK01

- Tandai kata benda pada deskripsi sebagai kandidat kelas, dan kata kerja sebagai kandidat operasi.
- Saring kandidat kelas dengan mempertimbangkan tanggung jawabnya, sehingga tidak ada peran yang tumpang tindih.
- Susun class diagram lengkap dengan atribut, operasi, tipe relasi, dan multiplisitas.
- Susun sequence diagram untuk sekurang-kurangnya dua skenario utama sistem.
- Tentukan titik rancangan yang memakai antarmuka agar bagian yang berubah terpisah dari yang stabil, lalu tuliskan alasannya.
- Terjemahkan class diagram menjadi kerangka kelas Java — isi metode boleh dikosongkan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kriteria dan Bobot Penilaian LK01

Diambil langsung dari berkas Lembar Kerja

Kriteria	Bobot	Yang dinilai
Kebenaran class diagram	40%	Kelengkapan kelas, atribut, operasi, ketepatan tipe relasi, dan multiplisitas
Kebenaran sequence diagram	25%	Kesesuaian urutan interaksi dengan skenario dan konsistensinya dengan class diagram
Justifikasi keputusan perancangan	20%	Ketajaman alasan pemilihan komposisi atau pewarisan dan penggunaan antarmuka
Kesesuaian kerangka kode Java	15%	Kesesuaian struktur kelas Java dengan class diagram yang disusun

© Fakultas Ilmu Komputer, Universitas Brawijaya

Jadwal Pelaksanaan LK01

Pengumpulan paling lambat pukul 23.59 pada hari terakhir

Waktu	Aktivitas	Luaran
Pekan ke-3	Penjelasan tugas dan pembagian deskripsi sistem	Catatan pemahaman kasus
Pekan ke-3 s.d. ke-4	Penyusunan diagram dan kerangka kode secara mandiri	Draf diagram dan kerangka kelas
Akhir pekan ke-4	Pengumpulan luaran melalui LMS BRONE	Laporan, berkas diagram, dan tautan repositori

Keterlambatan dikenai pengurangan nilai 10% per hari dan tidak diterima setelah tiga hari. Penggunaan kecerdasan artifisial generatif wajib dicantumkan pada laporan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- Program to an interface, not an implementation menuntut klien bergantung pada kontrak, bukan pada kelas tertentu.
- Coupling mengukur ketergantungan antarmodul dan cohesion mengukur keterkaitan di dalam modul; keduanya dinilai bersama.
- Class diagram menyatakan struktur rancangan; sequence diagram menyatakan alur interaksi objek pada satu skenario.
- Kedua diagram wajib konsisten: setiap pesan pada sequence diagram harus ada sebagai operasi pada class diagram.
- Tugas 1 (LK01) mengukur ketiga hal di atas sekaligus, dan dikumpulkan pada akhir pekan ke-4.

Pekan depan: **prinsip SOLID bagian pertama — Single Responsibility, Open/Closed, dan Liskov Substitution.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Gamma, E., Helm, R., Johnson, R. dan Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, hlm. 11–22.
- Refactoring Guru, 2024. Design Patterns. [daring] Tersedia pada: <https://refactoring.guru/design-patterns> [Diakses 5 Juli 2024].
- Freeman, E., Robson, E., Bates, B. dan Sierra, K., 2004. Head First Design Patterns: A Brain-Friendly Guide. Sebastopol, CA: O'Reilly Media.
- Martin, R.C., 2002. Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall.

© Fakultas Ilmu Komputer, Universitas Brawijaya